Exp No: 6
10/9/26

HAMMING CODE

Aim:

Write a program to implement error detection and correction using HAMMING code concept. Make a test run to input data stream and verify error correction feature.

HAMMING CODE :

```
def sender():
    data = input("Enter text:")

    binary = ""
    for ch in data:
        binary += format(ord(ch), '08b')

    print("Binary data:", binary)
    m = len(binary)
    r = 0
    while 2**r < m + r + 1:
        r += 1
    n = m + r
    code = ['0'] * (n+1)
    j = 0
    for i in range(1, n+1):
        if i & (i-1):
            code[i] = binary[j]
            j += 1
    # calculate parity bits
    for p in range(r):
        pos = 2**p
        parity = 0
        for i in range(1, n+1):
            if i & pos
```

```
            parity ^= int(code[i])
        code[pos] = str[parity]
    hamming = "".join(code[i:])
    print("Hamming code:", hamming)
        # save to channel
    with open("channel", "w") aa f:
        f.write(hamming)
```

Sender:

```
# r1, r2, r4 and r8 are redundancy bits
text = input("Enter the text:")
    # convert text to binary
    binary = ""
for ch in text
    binary += format(ord(ch), '08b')
print("Binary data", binary)
    channel = ""
    # process 8 data bits at a time
    for x in range(0, len(binary), 8):
        d = binary[xx+8]
        # 12 bit Hamming code
        # position : 1 2 3 4 5 6
        code = ['0'] * 13
        code = [3] = d[0]
        code[6] = d[2]
        code[7] = d[3]
```

```
code[9] = d[4]
code[10] = d[5]
code[11] = d[6]
code[12] = d[7]
code[i] = str(int(code[3]) ^ int(code[5])
        ^ int(code[7]) ^ int(code[9]) ^
          int(code[11]))
code[2] = str(int(code[5]) ^ int(code
    [6]) ^ int(code[7]) ^ int(code[10]) ^
      int(code[11]))
code[4] = str(int(code[5]) ^ int(code[6]
    ) ^ int(code[7]) ^ int(code[12]))
code[8] = str(int(code[9]) ^ int(code
    [10]) ^ int(code[11]) ^ int(code[12]))

for i in range(1,13):
    channel += code[i]
print("Hamming code:", "r") as f:
with open("channel:txt", "w")
    f.write(channel)
```

Receiver:

```
with open("channel.txt", "r") as f:
    received = f.read()
print("Received data:", received)
    corrected = ""

for x in range(0, len(received), 12):
        c = ['0'] + list(received
            [x:x+12])
```

```
r1 = (int(c[1]) ^ int(c[3]) ^ int(c[5]) ^
      int(c[7]) ^ int(c[9]) ^
      int(c[11])
r2 = (int(c[2]) ^ int(c[3]) ^ int(c[6]) ^
      int(c[7]) ^ int(c[10]) ^ int(c[11]))
r4 = (int(c[4]) ^ int(c[5]) ^ int[c[6]) ^
      int(c[7]) ^ int(c[12]))
r8 = (int(c[8]) ^ int(c[9]) ^ int(c[10]) ^
      int(c[11]) ^ int(c[12]))
error = r1*1 + r2*2 + r4*4 + r8*8
if error == 0:
    print("No error detected")
else:
    print("Error detected at position:", error)
    if c[error] == '0':
        c[error] = '1'
    else:
        c[error] = '0'
    print("Error corrected.")
corrected += (c[3] + c[5] + c[6] + c[7] +
              [c[9] + c[10] + c[11] + c[12])
output = ""
for x in range(0, len corrected), 8):
    byte = corrected[x : x+8]
    if(len(byte) == 8:
        output += chr(int(byte, 2))
print("original text :", output)
```

Output :

Enter the text = hello

Binary data : 01101000011001010110110001101111

Hamming code : 01001101110001001111000101000011
00110000000110011001101111001111

Hamming code saved in channel.txt :

Received data : 01000101100010011110001010000
11001100000001100110011011110011

Error detected in position : 5

Error corrected.